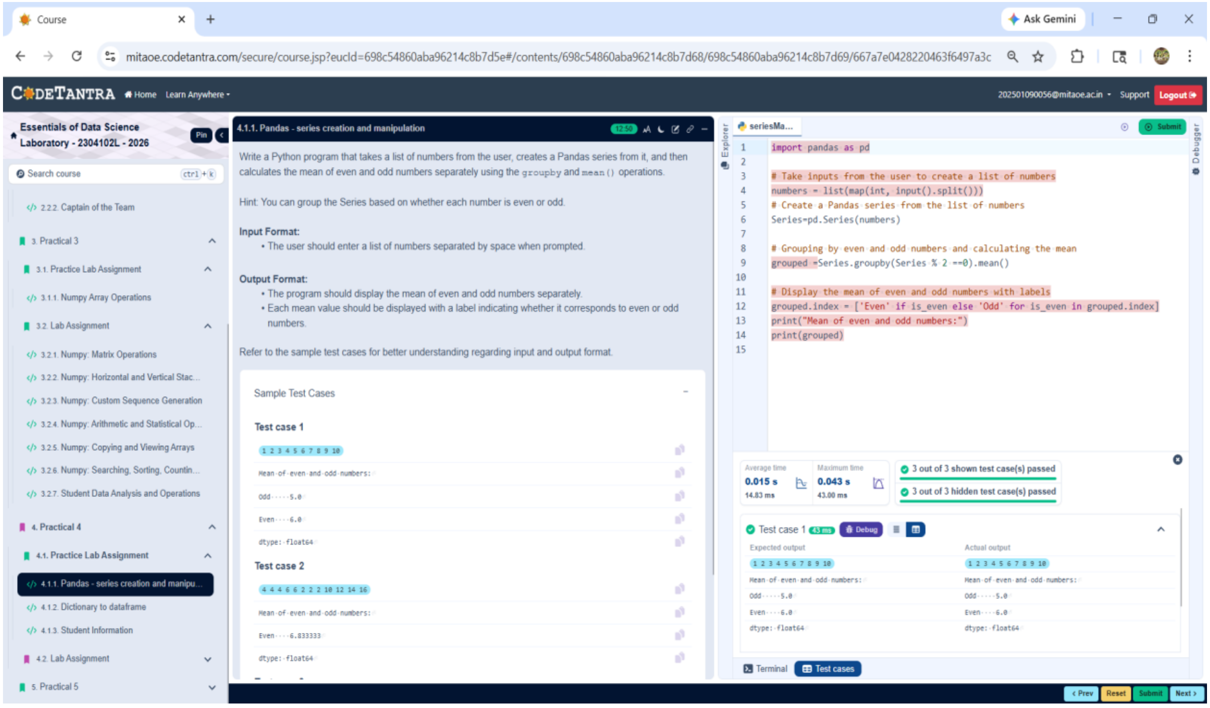
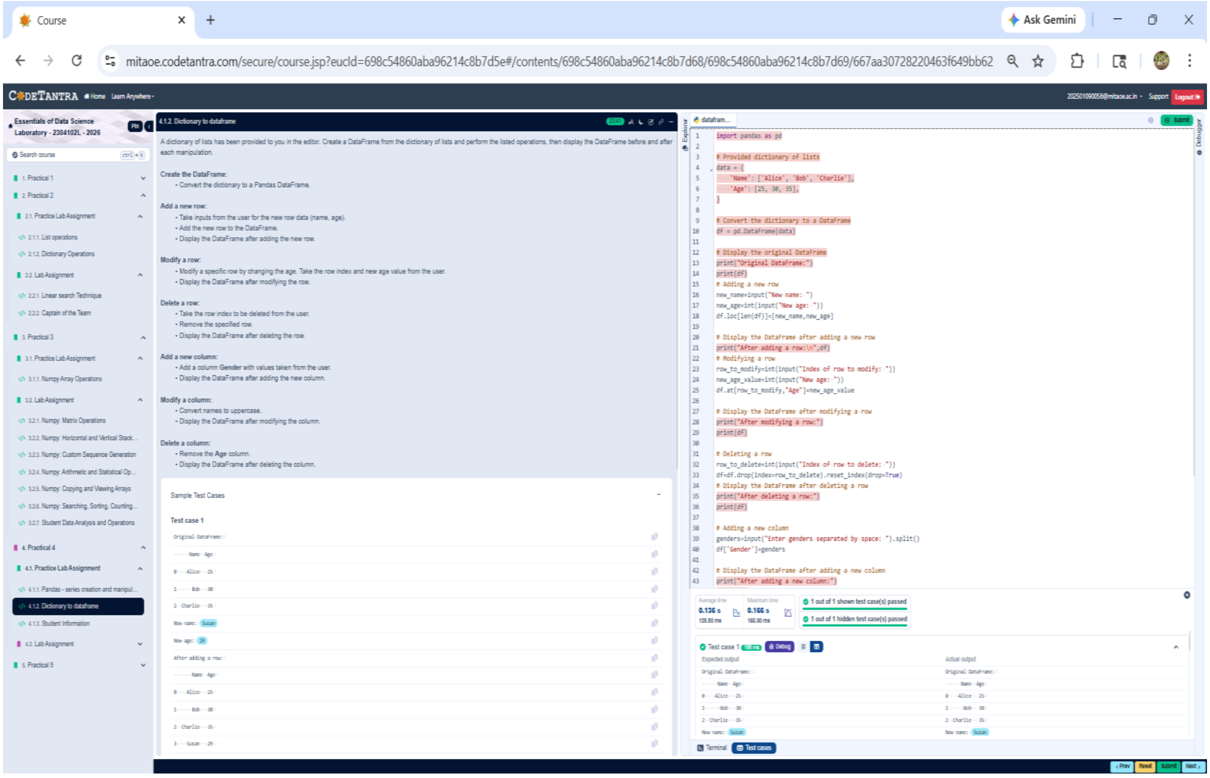
Codetantra Practical 4

Practice Lab Assignment





Course
Ask Gemini
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d68/698c54860aba96214c8b7d69/667aa30728220463f649bb62
Essentials of Data Science Laboratory - 2304102L - 2026
4.1.2. Dictionary to dataframe
A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.
Create the DataFrame:
• Convert the dictionary to a Pandas DataFrame.
Add a new row:
• Take inputs from the user for the new row data (name, age).
• Add the new row to the DataFrame.
• Display the DataFrame after adding the new row.
Modify a row:
• Modify a specific row by changing the age. Take the row index and new age value from the user.
• Display the DataFrame after modifying the row.
Delete a row:
• Take the row index to be deleted from the user.
• Remove the specified row.
• Display the DataFrame after deleting the row.
Add a new column:
• Add a column Gender with values taken from the user.
• Display the DataFrame after adding the new column.
Modify a column:
• Convert names to uppercase.
• Display the DataFrame after modifying the column.
Delete a column:
• Remove the Age column.
• Display the DataFrame after deleting the column.
Sample Test Cases
Test case 1
Original DataFrame:
Name Age
0 Alice 25
1 Bob 30
2 Charlie 35
New name: Susan
New age: 28
After adding a row:
Name Age
0 Alice 25
1 Bob 30
2 Charlie 35
3 Susan 28
import pandas as pd
# Provided dictionary of lists
data = {
'Name': ['Alice', 'Bob', 'Charlie'],
'Age': [25, 30, 35],
}
# Convert the dictionary to a DataFrame
df = pd.DataFrame(data)
# Display the original DataFrame
print("Original DataFrame:")
print(df)
# Adding a new row
new_name=input("New name: ")
new_age=int(input("New age: "))
df.loc[len(df)]=[new_name,new_age]
# Display the DataFrame after adding a new row
print("After adding a row:\n",df)
# Modifying a row
row_to_modify=int(input("Index of row to modify: "))
new_age_value=int(input("New age: "))
df.at[row_to_modify,"Age"]=new_age_value
# Display the DataFrame after modifying a row
print("After modifying a row:")
print(df)
# Deleting a row
row_to_delete=int(input("Index of row to delete: "))
df=df.drop(index=row_to_delete).reset_index(drop=True)
# Display the DataFrame after deleting a row
print("After deleting a row:")
print(df)
# Adding a new column
genders=input("Enter genders separated by space: ").split()
df['Gender']=genders
# Display the DataFrame after adding a new column
print("After adding a new column:")
0.136 s
0.166 s
1 out of 1 shown test case(s) passed
1 out of 1 hidden test case(s) passed
Test case 1
Expected output
Actual output
Terminal
Test cases

Essentials of Data Science Laboratory - 2304102L - 2026

4.1.3. Student Information

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is "**B**").

Note:
Refer to the displayed test cases for better understanding.

Sample Test Cases

Test case 1

```
studentdata.txt
First five rows:
   Name  Age Grade
0  John   25     A
1  Alin   22     B
2  Emma   24     A
3  Mike   23     C
4  Sony   21     B
Average age: 23.29
Students with a grade up to B
   Name  Age Grade
0  John   25     A
```

studentin...

```python
import pandas as pd

# Read the text file into a DataFrame
file = input()
data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age", "Grade"])
print("First five rows:")
print(data.head())
ave=round(data["Age"].mean(),2)
print("Average age:",ave)
filtered_data=data[data['Grade'].isin(['A','B'])]
print("Students with a grade up to B")
print(filtered_data)
```

Average time 0.069 s 68.50 ms
Maximum time 0.100 s 100.00 ms

1 out of 1 shown test case(s) passed
1 out of 1 hidden test case(s) passed

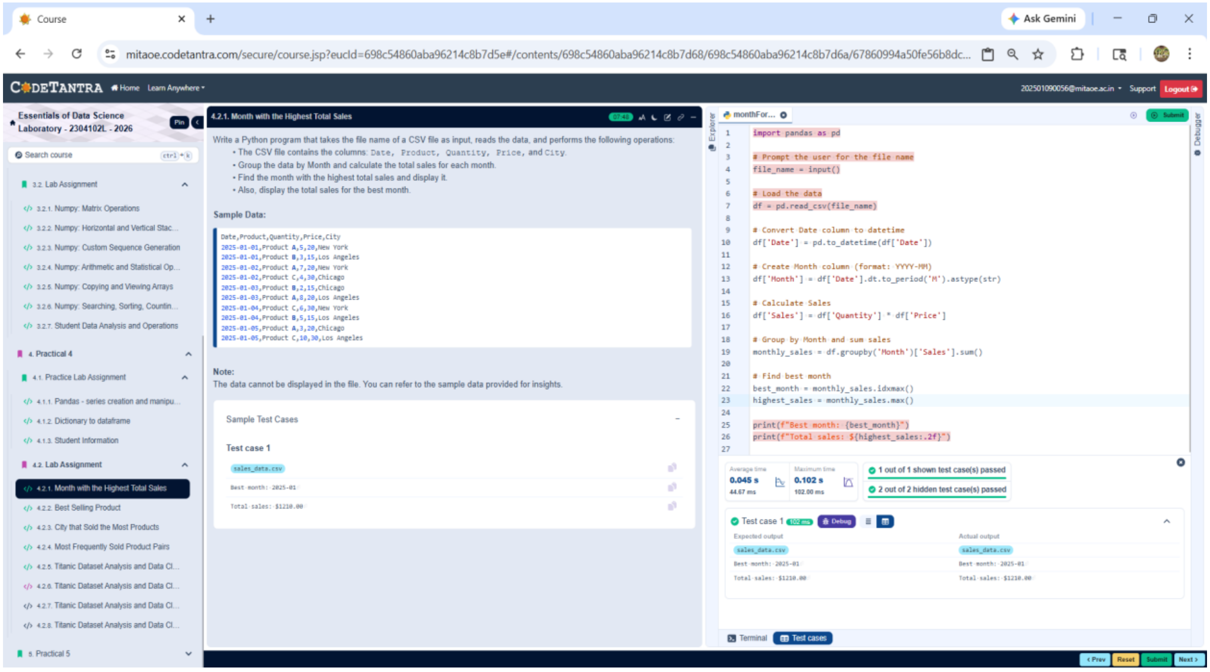
Test case 1 100 ms Debug

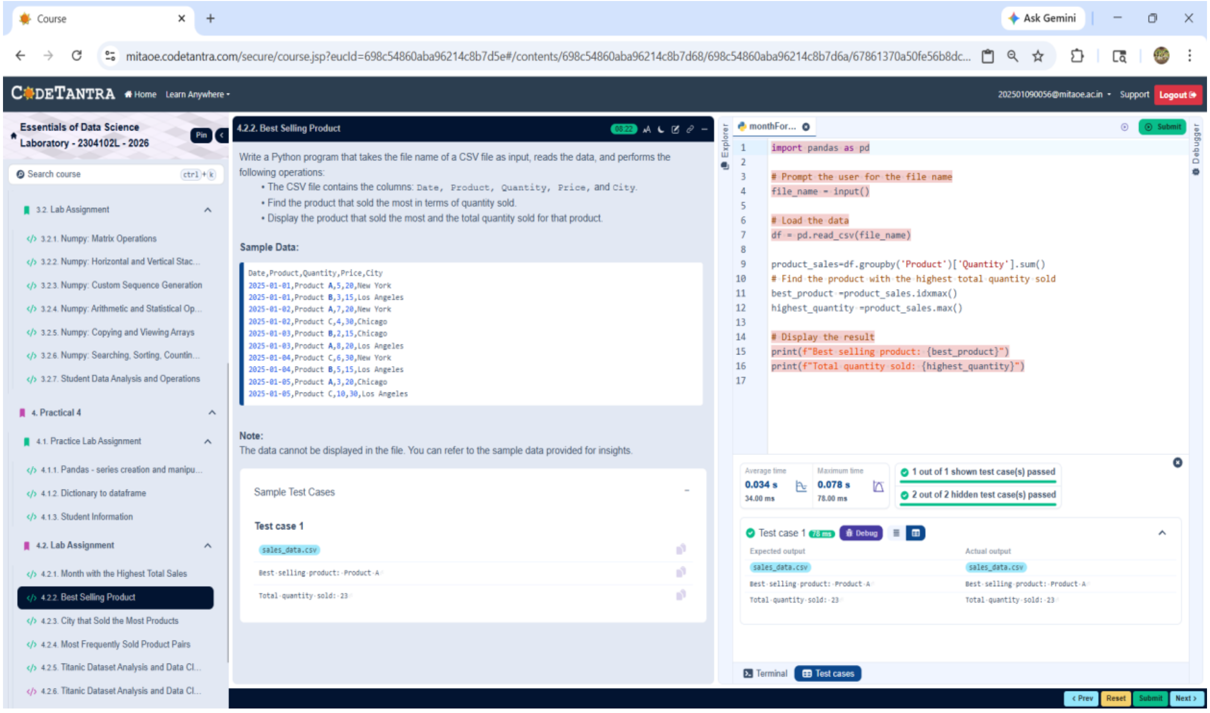
Expected output	Actual output
studentdata.txt	studentdata.txt
First five rows:	First five rows:
Name Age Grade	Name Age Grade
0 John 25 A	0 John 25 A
1 Alin 22 B	1 Alin 22 B
2 Emma 24 A	2 Emma 24 A

Terminal Test cases

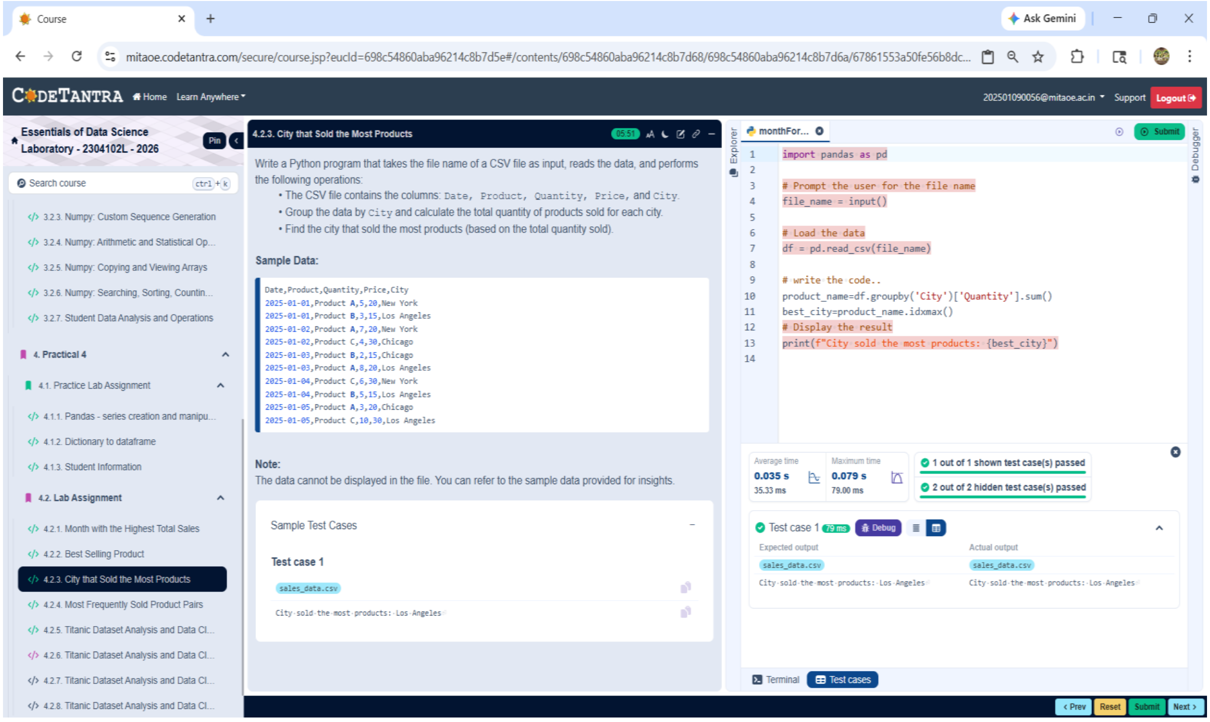
‹ Prev Reset Submit Next ›

Lab Assignment





Course
Ask Gemini
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d68/698c54860aba96214c8b7d6a/67861370a50fe56b8dc...
CODETANTRA
Home
Learn Anywhere
202501090056@mitaoe.ac.in
Support
Logout
Essentials of Data Science Laboratory - 2304102L - 2026
Pin
Search course
ctrl + k
3.2. Lab Assignment
3.2.1. Numpy: Matrix Operations
3.2.2. Numpy: Horizontal and Vertical Stac...
3.2.3. Numpy: Custom Sequence Generation
3.2.4. Numpy: Arithmetic and Statistical Op...
3.2.5. Numpy: Copying and Viewing Arrays
3.2.6. Numpy: Searching, Sorting, Countin...
3.2.7. Student Data Analysis and Operations
4. Practical 4
4.1. Practice Lab Assignment
4.1.1. Pandas - series creation and manipu...
4.1.2. Dictionary to dataframe
4.1.3. Student Information
4.2. Lab Assignment
4.2.1. Month with the Highest Total Sales
4.2.2. Best Selling Product
4.2.3. City that Sold the Most Products
4.2.4. Most Frequently Sold Product Pairs
4.2.5. Titanic Dataset Analysis and Data Cl...
4.2.6. Titanic Dataset Analysis and Data Cl...
4.2.2. Best Selling Product
08:22
Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:
• The CSV file contains the columns: Date, Product, Quantity, Price, and City.
• Find the product that sold the most in terms of quantity sold.
• Display the product that sold the most and the total quantity sold for that product.
Sample Data:
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.
Sample Test Cases
Test case 1
sales_data.csv
Best selling product: Product A
Total quantity sold: 23
Explorer
monthFor...
Submit
Debugger
import pandas as pd
# Prompt the user for the file name
file_name = input()
# Load the data
df = pd.read_csv(file_name)
product_sales=df.groupby('Product')['Quantity'].sum()
# Find the product with the highest total quantity sold
best_product =product_sales.idxmax()
highest_quantity =product_sales.max()
# Display the result
print(f"Best selling product: {best_product}")
print(f"Total quantity sold: {highest_quantity}")
Average time
0.034 s
34.00 ms
Maximum time
0.078 s
78.00 ms
1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
78 ms
Debug
Expected output
sales_data.csv
Best selling product: Product A
Total quantity sold: 23
Actual output
sales_data.csv
Best selling product: Product A
Total quantity sold: 23
Terminal
Test cases
Prev
Reset
Submit
Next


Essentials of Data Science Laboratory - 2304102L - 2026
3.2.3. Numpy: Custom Sequence Generation
3.2.4. Numpy: Arithmetic and Statistical Op...
3.2.5. Numpy: Copying and Viewing Arrays
3.2.6. Numpy: Searching, Sorting, Countin...
3.2.7. Student Data Analysis and Operations
4. Practical 4
4.1. Practice Lab Assignment
4.1.1. Pandas - series creation and manipu...
4.1.2. Dictionary to dataframe
4.1.3. Student Information
4.2. Lab Assignment
4.2.1. Month with the Highest Total Sales
4.2.2. Best Selling Product
4.2.3. City that Sold the Most Products
4.2.4. Most Frequently Sold Product Pairs
4.2.5. Titanic Dataset Analysis and Data Cl...
4.2.6. Titanic Dataset Analysis and Data Cl...
4.2.7. Titanic Dataset Analysis and Data Cl...
4.2.8. Titanic Dataset Analysis and Data Cl...
4.2.3. City that Sold the Most Products
Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:
• The CSV file contains the columns: Date, Product, Quantity, Price, and City.
• Group the data by City and calculate the total quantity of products sold for each city.
• Find the city that sold the most products (based on the total quantity sold).
Sample Data:
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.
Sample Test Cases
Test case 1
sales_data.csv
City sold the most products: Los Angeles
import pandas as pd
# Prompt the user for the file name
file_name = input()
# Load the data
df = pd.read_csv(file_name)
# write the code..
product_name=df.groupby('City')['Quantity'].sum()
best_city=product_name.idxmax()
# Display the result
print(f"City sold the most products: {best_city}")
Average time 0.035 s 35.33 ms
Maximum time 0.079 s 79.00 ms
1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1 79 ms
Expected output
sales_data.csv
City sold the most products: Los Angeles
Actual output
sales_data.csv
City sold the most products: Los Angeles

4.2.4. Most Frequently Sold Product Pairs

02:17

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Explanation:
Transactions:

- **2025-01-01:** Product A, Product B
- **2025-01-02:** Product A, Product C
- **2025-01-03:** Product B, Product A
- **2025-01-04:** Product C, Product B
- **2025-01-05:** Product A, Product C

Now, let's count how often the pairs of products appear together:

- **Product A and Product B**: Appear in transactions on 2025-01-01 and 2025-01-03.
- **Product A and Product C**: Appear in transactions on 2025-01-02 and 2025-01-05.
- **Product B and Product C**: Appears in transactions on 2025-01-04.

Most Frequent Product Combinations:

- **Product A and Product B** (2 times)
- **Product A and Product C** (2 times)

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

Test case 1

sales_data.csv

frequentl...

```python
import pandas as pd
from itertools import combinations
from collections import Counter

# Prompt user to input the file name
file_name = input()

# Read data from the specified CSV file
df = pd.read_csv(file_name)
# Group products by Date (transactions)
grouped = df.groupby('Date')['Product'].apply(list)
# Generate pairs
pair_counts = Counter()
for products in grouped:
    pairs = combinations(sorted(products), 2)
    pair_counts.update(pairs)

# Find maximum frequency
max_count = max(pair_counts.values())
# Get most frequent pairs
most_common_pairs = [pair for pair, count in pair_counts.items() if count == max_count]

# Sort pairs alphabetically
most_common_pairs.sort()
# Output
for pair in most_common_pairs:
    print(f"{pair[0]} and {pair[1]}: {max_count} times")
```

Average time 0.021 s 21.33 ms | Maximum time 0.035 s 35.00 ms

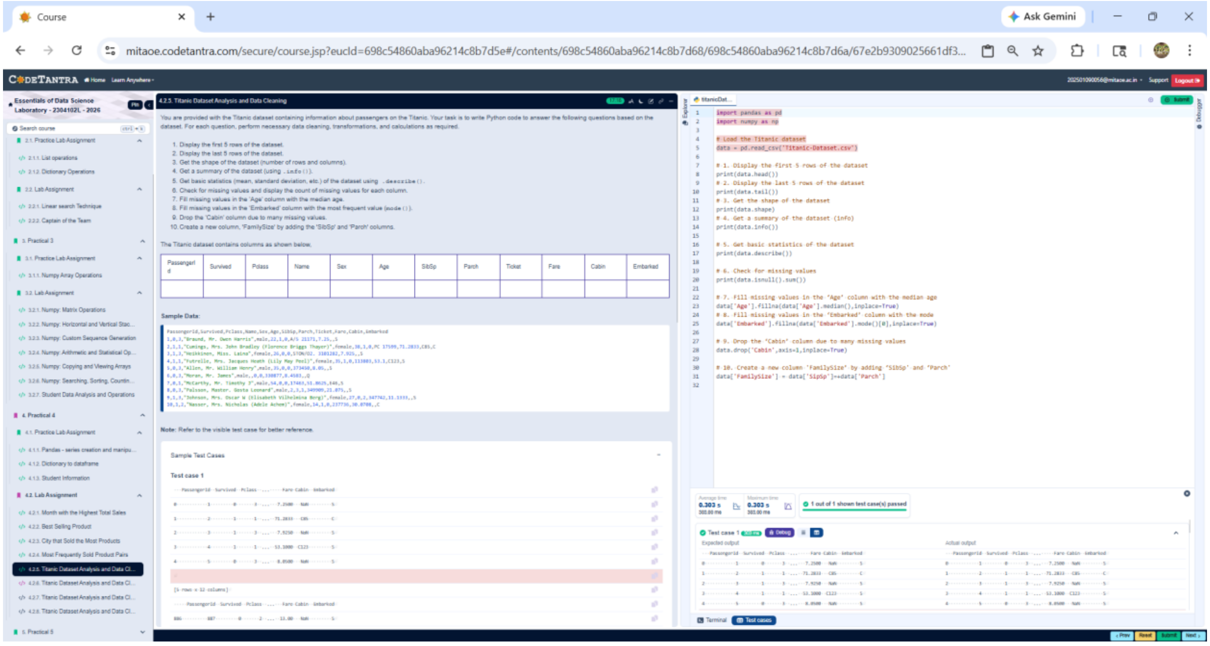
1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed

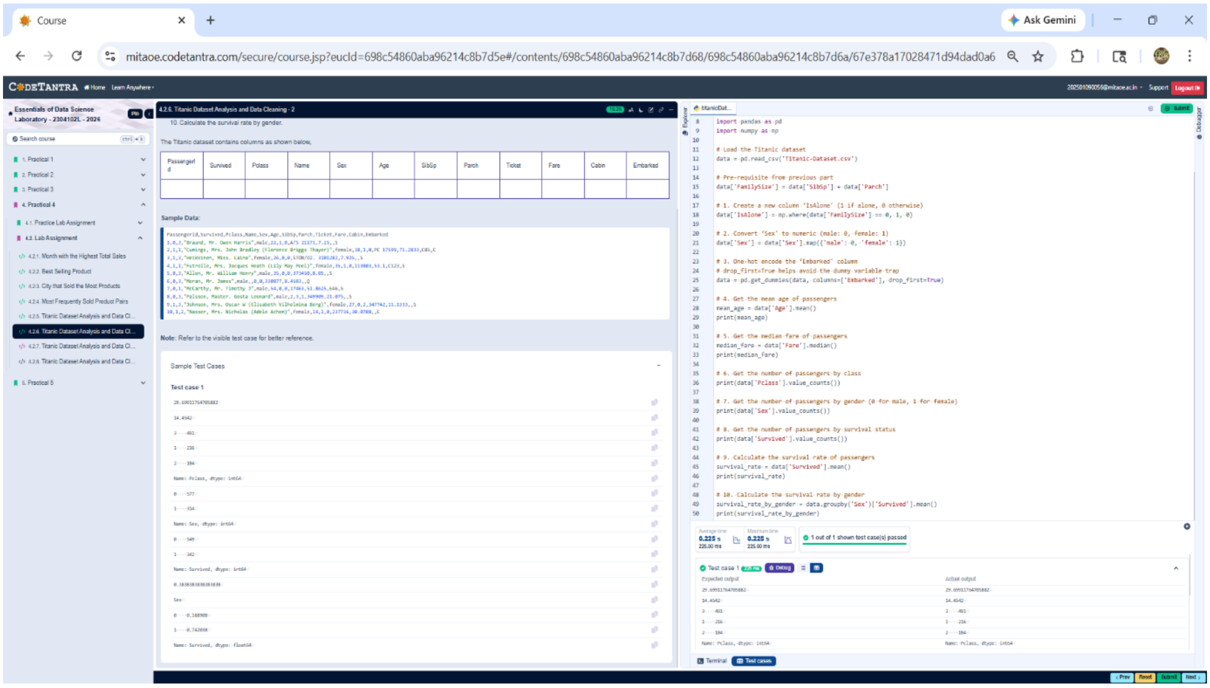
Test case 1 35 ms Debug

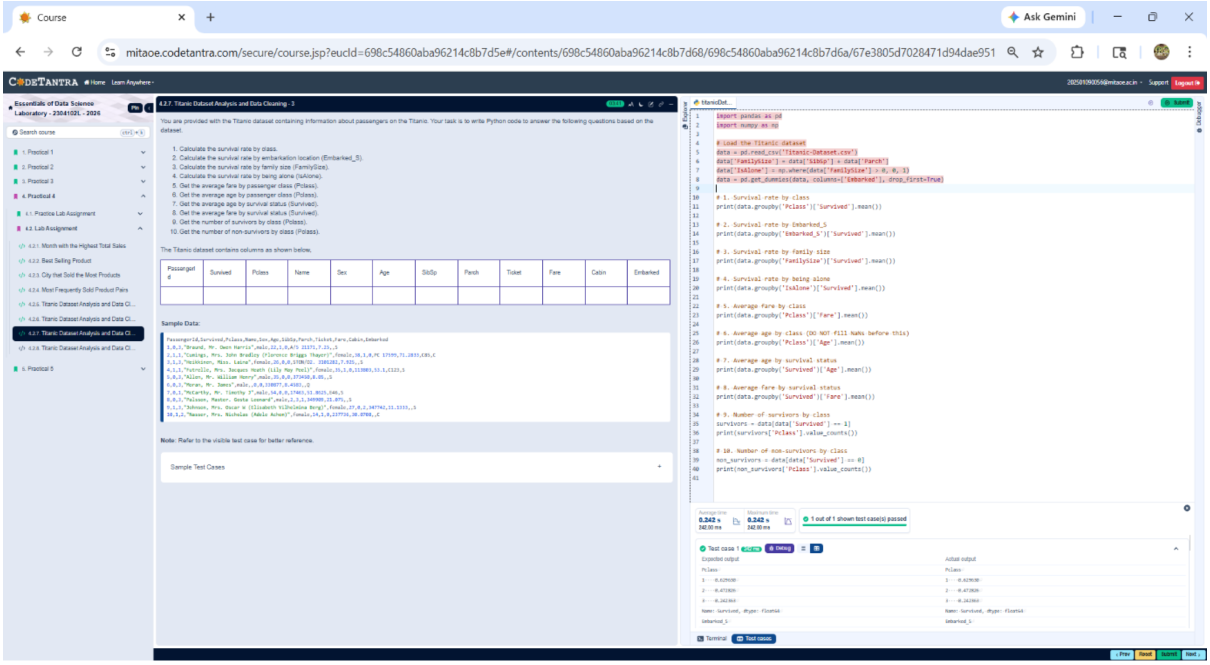
Expected output	Actual output
sales_data.csv	sales_data.csv
Product A and Product B: 2 times	Product A and Product B: 2 times
Product A and Product C: 2 times	Product A and Product C: 2 times

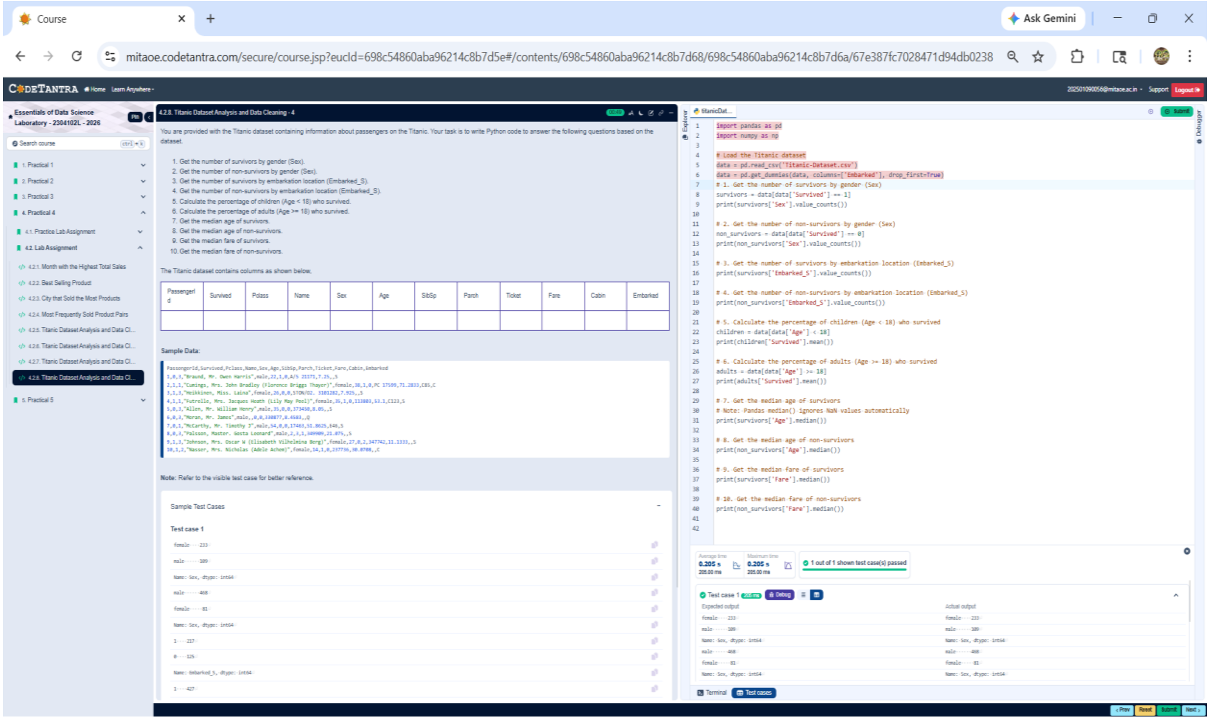
Terminal Test cases

< Prev Reset Submit Next >


Course
Ask Gemini
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d68/698c54860aba96214c8b7d6a/67e2b9309025661df3...
Support
Logout
Essentials of Data Science Laboratory - 2304102L - 2026
4.2.5. Titanic Dataset Analysis and Data Cleaning
You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.
1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using .info()).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using .describe().
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (mode()).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.
The Titanic dataset contains columns as shown below.
PassengerId Survived Pclass Name Sex Age SibSp Parch Ticket Fare Cabin Embarked
Sample Data:
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
Note: Refer to the visible test case for better reference.
Sample Test Cases
Test case 1
titanicDat...
import pandas as pd
import numpy as np
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
# 1. Display the first 5 rows of the dataset
print(data.head())
# 2. Display the last 5 rows of the dataset
print(data.tail())
# 3. Get the shape of the dataset
print(data.shape)
# 4. Get a summary of the dataset (info)
print(data.info())
# 5. Get basic statistics of the dataset
print(data.describe())
# 6. Check for missing values
print(data.isnull().sum())
# 7. Fill missing values in the 'Age' column with the median age
data['Age'].fillna(data['Age'].median(),inplace=True)
8. Fill missing values in the 'Embarked' column with the mode
data['Embarked'].fillna(data['Embarked'].mode()[0],inplace=True)
# 9. Drop the 'Cabin' column due to many missing values
data.drop('Cabin',axis=1,inplace=True)
# 10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch'
data['FamilySize'] = data['SipSp']+data['Parch']
1 out of 1 shown test case(s) passed
Expected output
Actual output
Terminal
Test cases


Course
Ask Gemini
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d68/698c54860aba96214c8b7d6a/67e378a17028471d94dad0a6
Learn Anywhere
Essentials of Data Science Laboratory - 23041021 - 2026
Search course
1. Practical 1
2. Practical 2
3. Practical 3
4. Practical 4
4.1. Practice Lab Assignment
4.2. Lab Assignment
4.2.1. Month with the Highest Total Sales
4.2.2. Best Selling Product
4.2.3. City that Sold the Most Products
4.2.4. Most Frequently Sold Product Pairs
4.2.5. Titanic Dataset Analysis and Data Cl...
4.2.6. Titanic Dataset Analysis and Data Cl...
4.2.7. Titanic Dataset Analysis and Data Cl...
4.2.8. Titanic Dataset Analysis and Data Cl...
5. Practical 5
4.2.6. Titanic Dataset Analysis and Data Cleaning - 2
10. Calculate the survival rate by gender.
The Titanic dataset contains columns as shown below.
PassengerId Survived Pclass Name Sex Age SibSp Parch Ticket Fare Cabin Embarked
Sample Data:
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
Note: Refer to the visible test case for better reference.
Sample Test Cases
Test case 1
29.69911764705882
14.4542
3 491
1 216
2 184
Name: Pclass, dtype: int64
0 577
1 314
Name: Sex, dtype: int64
0 549
1 342
Name: Survived, dtype: int64
0.3838383838383838
Sex
0 0.188908
1 0.742038
Name: Survived, dtype: float64
titanicDat...
Submit
import pandas as pd
import numpy as np
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
# Pre-requisite from previous part
data['FamilySize'] = data['SibSp'] + data['Parch']
1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
data['IsAlone'] = np.where(data['FamilySize'] == 0, 1, 0)
2. Convert 'Sex' to numeric (male: 0, female: 1)
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
# 3. One-hot encode the 'Embarked' column
# drop_first=True helps avoid the dummy variable trap
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
# 4. Get the mean age of passengers
mean_age = data['Age'].mean()
print(mean_age)
# 5. Get the median fare of passengers
median_fare = data['Fare'].median()
print(median_fare)
# 6. Get the number of passengers by class
print(data['Pclass'].value_counts())
# 7. Get the number of passengers by gender (0 for male, 1 for female)
print(data['Sex'].value_counts())
# 8. Get the number of passengers by survival status
print(data['Survived'].value_counts())
# 9. Calculate the survival rate of passengers
survival_rate = data['Survived'].mean()
print(survival_rate)
# 10. Calculate the survival rate by gender
survival_rate_by_gender = data.groupby('Sex')['Survived'].mean()
print(survival_rate_by_gender)
Average time 0.225 s 225.00 ms
Maximum time 0.225 s 225.00 ms
1 out of 1 shown test case(s) passed
Test case 1
Debug
Expected output
Actual output
Terminal
Test cases
Prev
Reset
Submit
Next


Course
Ask Gemini
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d68/698c54860aba96214c8b7d6a/67e3805d7028471d94dae951
CODETANTRA Home Learn Anywhere
202501090056@mitaoe.ac.in Support Logout
Essentials of Data Science Laboratory - 23041102L - 2026
4.2.7. Titanic Dataset Analysis and Data Cleaning - 3
You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.
1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).
The Titanic dataset contains columns as shown below.
PassengerId Survived Pclass Name Sex Age SibSp Parch Ticket Fare Cabin Embarked
Sample Data:
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
Note: Refer to the visible test case for better reference.
Sample Test Cases
import pandas as pd
import numpy as np
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']
data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
# 1. Survival rate by class
print(data.groupby('Pclass')['Survived'].mean())
# 2. Survival rate by Embarked_S
print(data.groupby('Embarked_S')['Survived'].mean())
# 3. Survival rate by family size
print(data.groupby('FamilySize')['Survived'].mean())
# 4. Survival rate by being alone
print(data.groupby('IsAlone')['Survived'].mean())
# 5. Average fare by class
print(data.groupby('Pclass')['Fare'].mean())
# 6. Average age by class (DO NOT fill NaNs before this)
print(data.groupby('Pclass')['Age'].mean())
# 7. Average age by survival status
print(data.groupby('Survived')['Age'].mean())
# 8. Average fare by survival status
print(data.groupby('Survived')['Fare'].mean())
# 9. Number of survivors by class
survivors = data[data['Survived'] == 1]
print(survivors['Pclass'].value_counts())
# 10. Number of non-survivors by class
non_survivors = data[data['Survived'] == 0]
print(non_survivors['Pclass'].value_counts())
Average time 0.242 s 242.00 ms
Maximum time 0.242 s 242.00 ms
1 out of 1 shown test case(s) passed
Test case 1
Debug
Expected output
Actual output
Terminal
Test cases
Prev Reset Submit Next



Course
mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d68/698c54860aba96214c8b7d6a/67e387fc7028471d94db0238
4.2.8. Titanic Dataset Analysis and Data Cleaning - 4
You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.
1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.
The Titanic dataset contains columns as shown below.
PassengerId
Survived
Pclass
Name
Sex
Age
SibSp
Parch
Ticket
Fare
Cabin
Embarked
Sample Data:
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
Note: Refer to the visible test case for better reference.
Sample Test Cases
Test case 1
import pandas as pd
import numpy as np
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
# 1. Get the number of survivors by gender (Sex)
survivors = data[data['Survived'] == 1]
print(survivors['Sex'].value_counts())
# 2. Get the number of non-survivors by gender (Sex)
non_survivors = data[data['Survived'] == 0]
print(non_survivors['Sex'].value_counts())
# 3. Get the number of survivors by embarkation location (Embarked_S)
print(survivors['Embarked_S'].value_counts())
# 4. Get the number of non-survivors by embarkation location (Embarked_S)
print(non_survivors['Embarked_S'].value_counts())
# 5. Calculate the percentage of children (Age < 18) who survived
children = data[data['Age'] < 18]
print(children['Survived'].mean())
# 6. Calculate the percentage of adults (Age >= 18) who survived
adults = data[data['Age'] >= 18]
print(adults['Survived'].mean())
# 7. Get the median age of survivors
# Note: Pandas median() ignores NaN values automatically
print(survivors['Age'].median())
# 8. Get the median age of non-survivors
print(non_survivors['Age'].median())
# 9. Get the median fare of survivors
print(survivors['Fare'].median())
# 10. Get the median fare of non-survivors
print(non_survivors['Fare'].median())
Average time 0.205 s 205.00 ms
Maximum time 0.205 s 205.00 ms
1 out of 1 shown test case(s) passed
Test case 1
Debug
Expected output
Actual output
female 233
male 109
Name: Sex, dtype: int64
Terminal
Test cases